ДИСЦИПЛИНА **Математическая логика и теория алгоритмов**
(полное наименование дисциплины без сокращений)

ИНСТИТУТ **информационных технологий**

КАФЕДРА **прикладной математики**
(полное наименование кафедры)

ВИД УЧЕБНОГО МАТЕРИАЛА **Лекция**
(в соответствии с пп.1-11)

ПРЕПОДАВАТЕЛЬ **Батенков Кирилл Александрович**
(фамилия, имя, отчество)

СЕМЕСТР **1, 2024-2025**
(указать семестр обучения, учебный год)

1. НЕЛИНЕЙНЫЕ АЛГОРИТМЫ

1.1. Разветвляющиеся алгоритмы, алгоритмы выбора их свойства, сферы применения

При создании алгоритмов часто возникает необходимость исполнение тех или иных действий поставить в зависимость от некоторого условия [28]. Такая возможность называется ветвлением. Блок-схема ветвления приведена на рисунке 36.

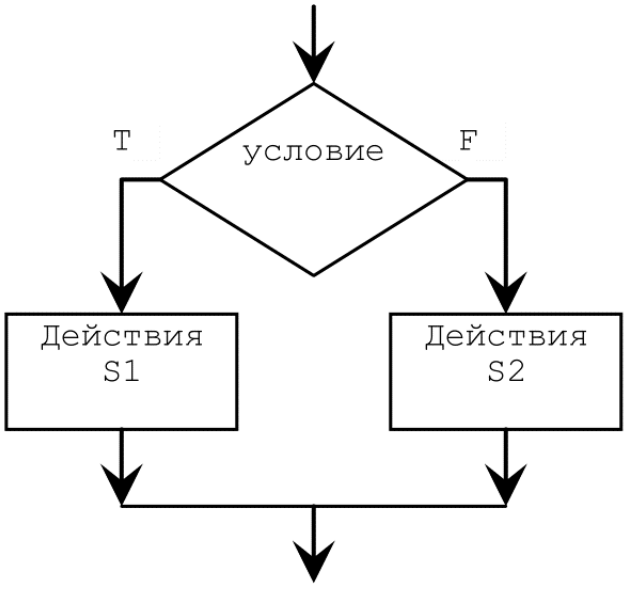


Рисунок 1. Алгоритм с ветвлением

Последовательность исполнения ветвления такова: вначале вычисляется значение условия - логического выражения. Затем, если значение условия истинно, то исполняются действия S1 (мы будем их всегда размещать на левой ветви ветвления и помечать эту ветвь буквой T); если же значение условия ложно, то исполняются действия S2 (такие действия будем располагать на правой ветви и помечать её буквой F).

Заметим, что при исполнении ветвления будет исполнена либо ветвь T (действия 51, слева), либо ветвь F (действия 52, справа). Одновременно пройти по обеим ветвям или не пройти ни по одной нельзя.

Иногда одна из ветвей алгоритма с ветвлением отсутствует, то есть нет либо действий S1, либо действий S2. Этот случай называют неполным ветвлением.

Составить блок-схему решения следующей задачи. Даны значения двух действительных переменных a и b. Найти наибольшее значение из a и b.

Составим блок-схему алгоритма по следующим соображениям. Мы должны сравнить значения переменных a и b, и если из них а имеет большее значение, то присвоить это значение переменной max. Если же a не больше b, но присвоить переменной max значение b. После этого в переменной max

будет храниться искомое наибольшее значение из a и b. Получаем блок-схему (рисунок 39).

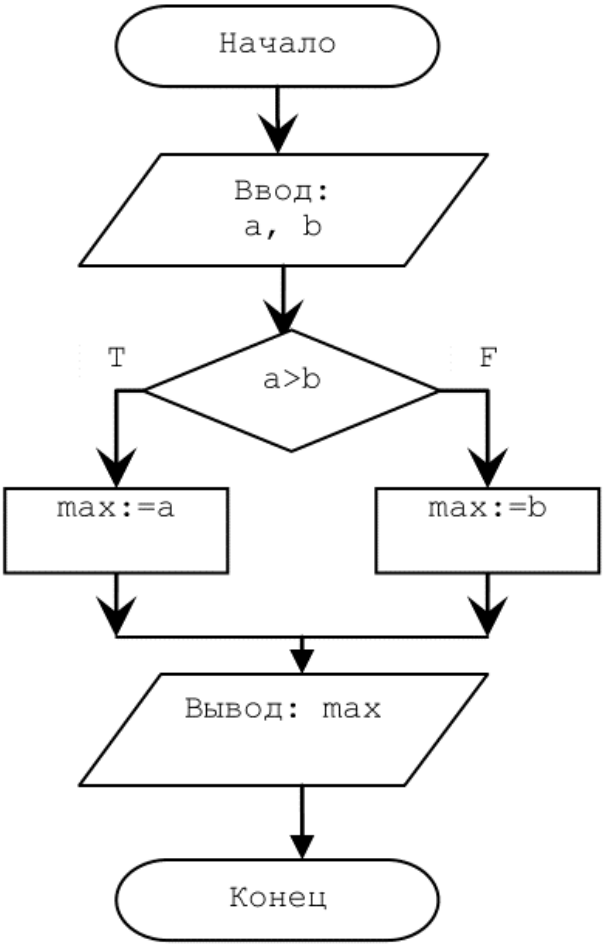


Рисунок 2. Нахождение наибольшего из a и b

Составить блок-схему решения следующей задачи. Даны значения трех действительных переменных a, b и c. Найти наибольшее значение из a, b и c.

Выбирать наибольшее значение из двух уже умеем (см. пример выше). В этой задаче надо найти большее из трех. Можно ли свести эту задачу к предыдущей? Можно, а именно – вначале найти наибольшее значение из a и b, а потом сравнив его и c, снова найти наибольшее из двух. Это можно представить такой блок-схемой (рис. 38).

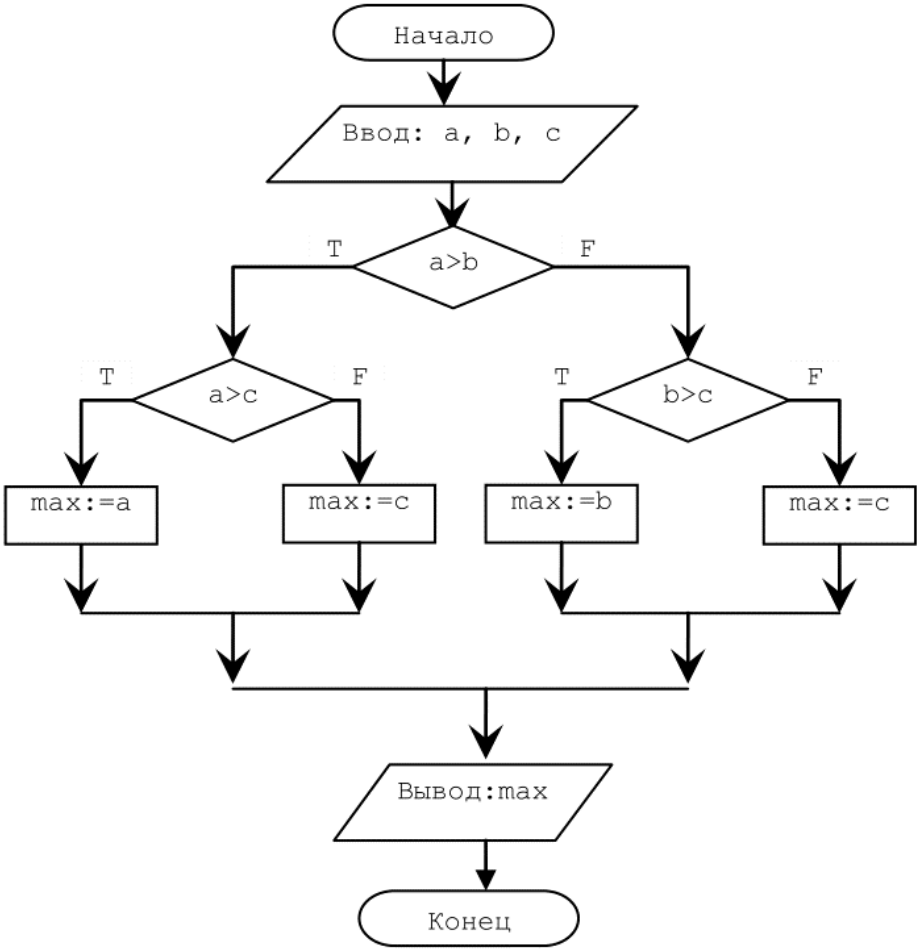


Рисунок 3. Алгоритм нахождения максимума из трех чисел

Составить блок-схему решения следующей задачи. Даны значения действительных переменных b и c. Решить линейное уравнение $bx + c = 0$.

Напомним, что решить уравнение – это найти множество всех его корней. Решение «в лоб» в виде «корень равен $-c/b$» неверно, так как не учитывает случай, когда $b = 0$, что вполне допустимо условием задачи. Действительно, если $b \neq 0$, то существует единственный корень, равный $-c/b$. Если же $b = 0$, то возможны два случая. Первый: если $c \neq 0$, то корней уравнение не имеет (то есть множество корней пусто $\emptyset$). Второй: если же $c = 0$, то уравнение не определено, то есть любое действительное число является его нем. Иначе говоря, множество корней уравнения в этом случае – все действительные числа $(-\infty;\ \infty)$.

Все вышесказанное можно представить в виде блок-схемы алгоритма (рис. 39).

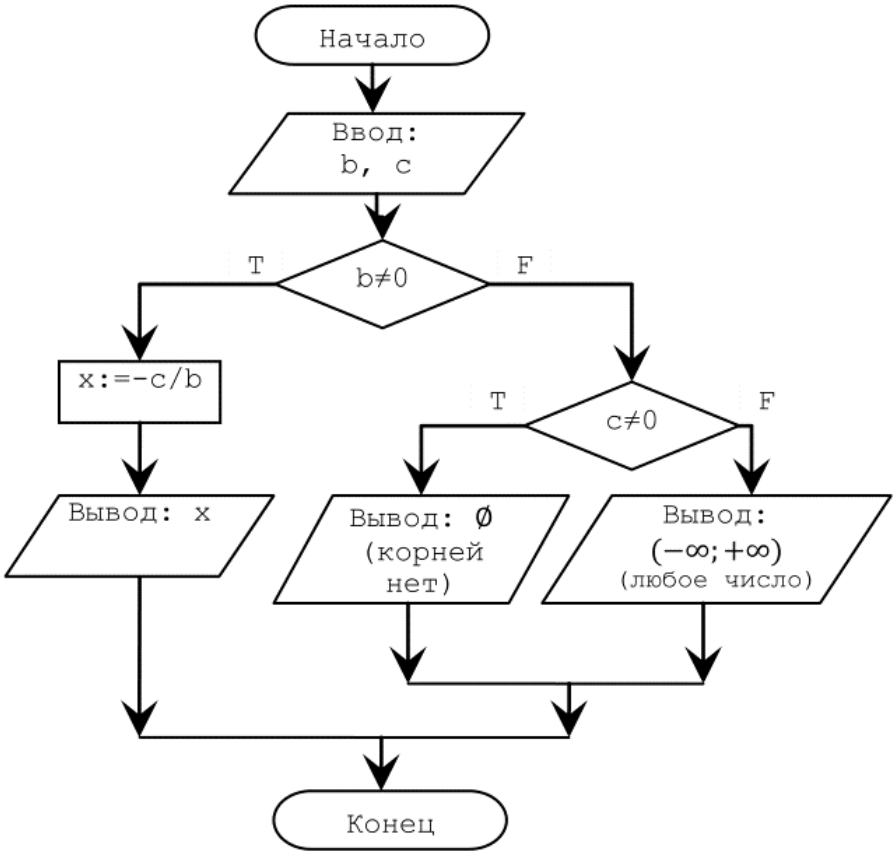


Рисунок 4. Алгоритм полного решения линейного уравнения

Составить блок-схему решения следующей задачи. Даны значения действительных переменных a, b, c, причем $a \neq 0$. Решить уравнение $ax^2 + bx + c = 0$.

Это квадратное уравнение, алгоритм его решения (через дискриминант) известен любому школьнику. Приведем его блок-схему (рис. 40). Обращаем внимание, что по условию этой задачи a не может равняться нулю.

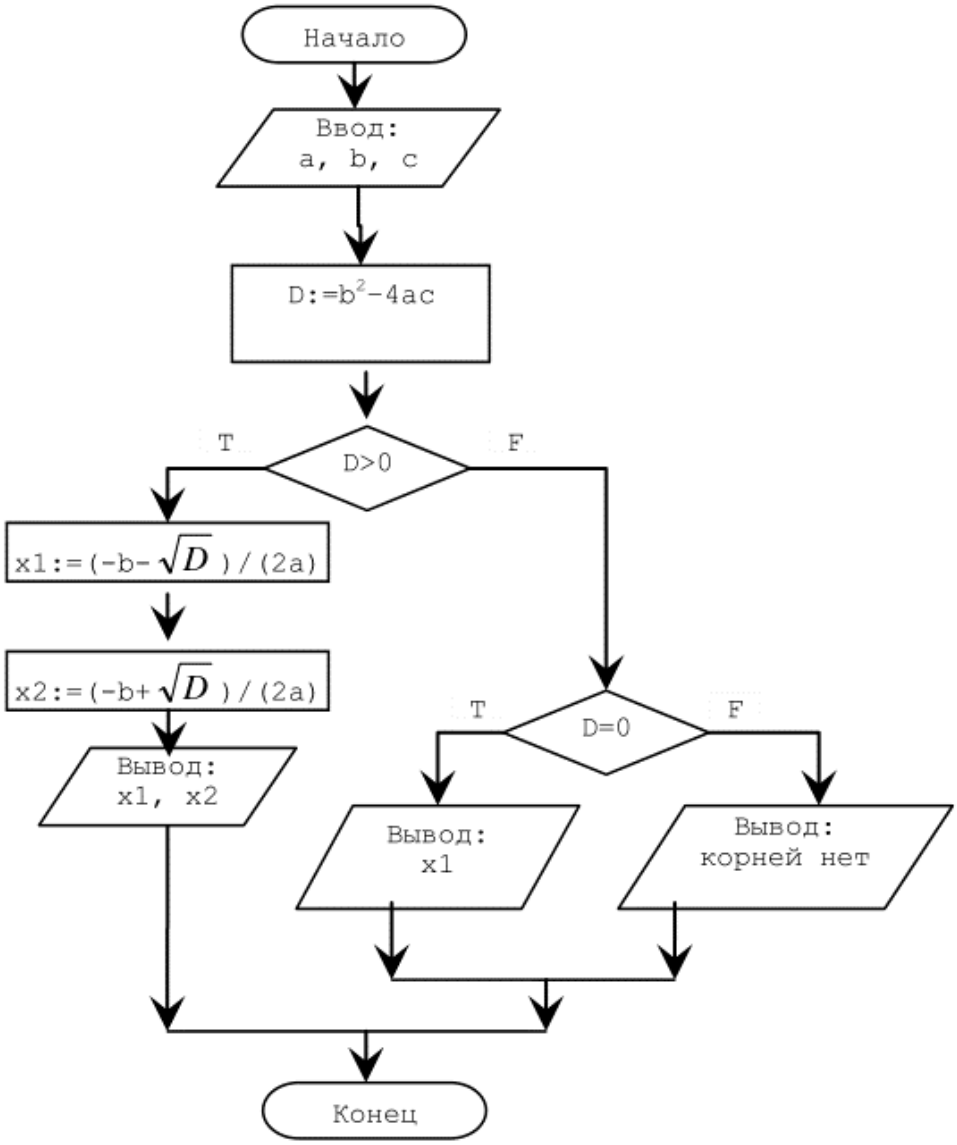


Рисунок 5. Алгоритм полного решения линейного уравнения

Для выбора из нескольких альтернативных действии используется алгоритмическая структура «выбор» [3030]. Перед выполнением команды «выбор» вычисляется значение некоторого выражения X, а затем начинается проверка условий $Y_1(X), Y_2(X) \dots Y_n(X)$. Проверка продолжается до тех пор. пока не встретится условие, принимающее значение истина при данном X (рис. 41).

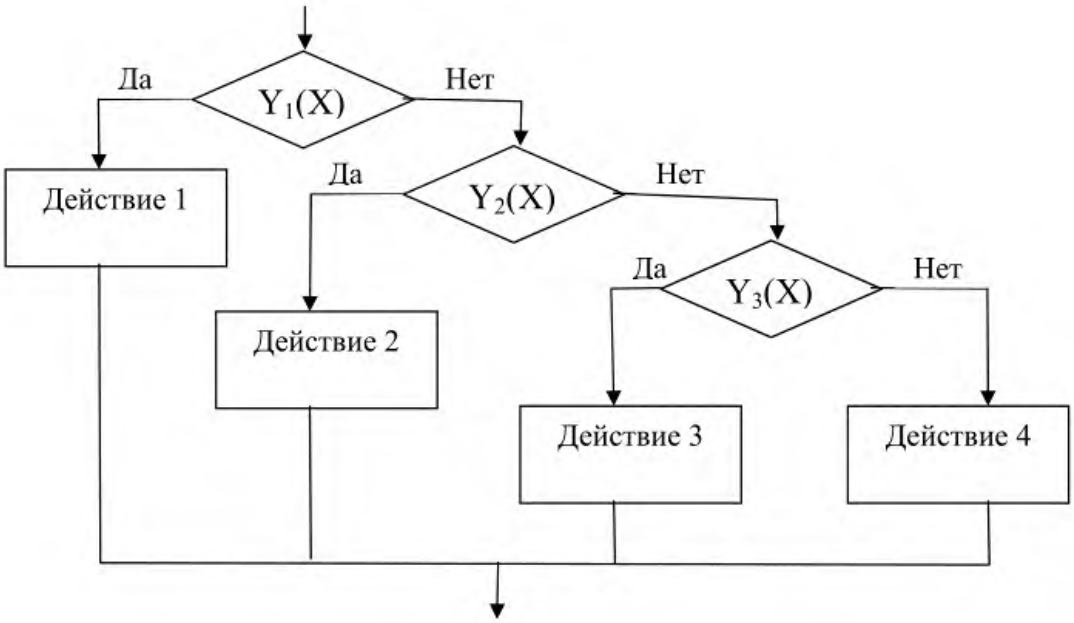


Рисунок 6. Алгоритмическая структура «выбор»

Структуру «выбор» можно также изобразить в форме, представленной на рис. 42.

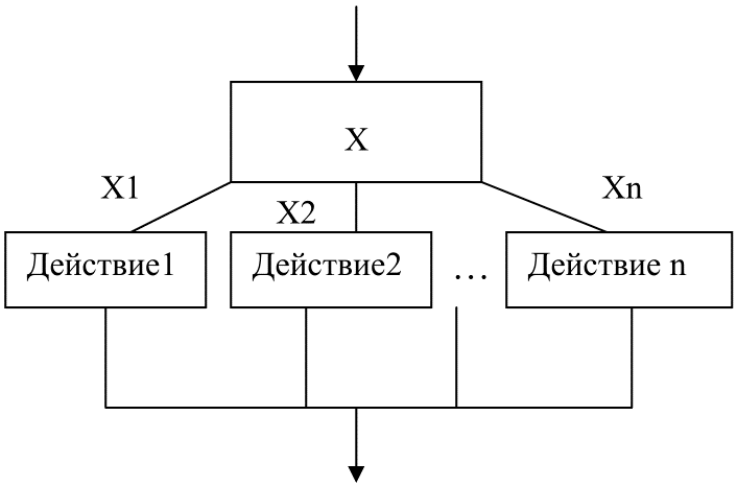


Рисунок 7. Вариант оформления алгоритмической структуры «выбор»

Составить блок-схему к программе, которая запрашивает у пользователя номер дня недели и выводит одно из сообщений: «рабочий день», «суббота» или «воскресенье» (рис. 43 и 44):

1. Ввод X.
2. Выбор X :

1…5: вывод «рабочий день»;

6: вывод «суббота»;

7: вывод «воскресенье»;

иначе вывод «Ошибка! Введите число от 1 до 7».

3. Конец.

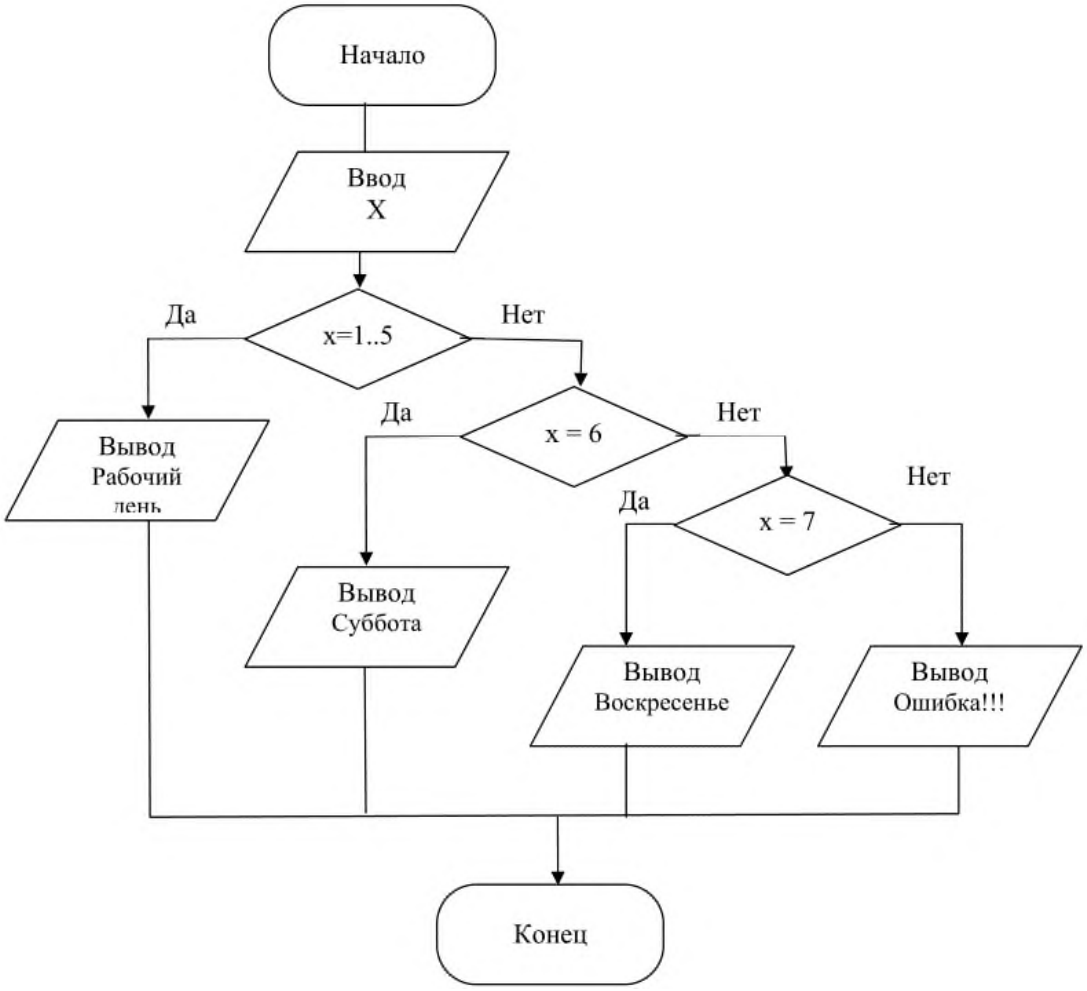


Рисунок 8. Блок-схема алгоритма определения дня недели

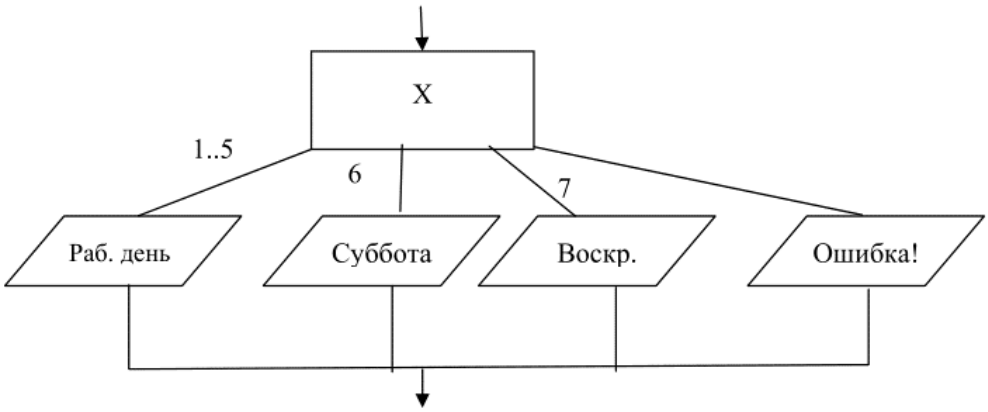


Рисунок 9. Блок-схема алгоритма определения дня недели типа «выбор»

1.2. Итерационные и циклические алгоритмы их свойства, сферы применения

Часто возникают ситуации, когда нужно совершить некоторую последовательность однообразных действий [28]. Например, чтобы забить гвоздь молотком, пока шляпка не будет утоплена, нужно ударять молотком по гвоздю (первый раз, второй раз, третий раз и т.д.). Алгоритмы, основанные на понятии многократного повторения некоторых действий называются *циклическими*.

Итак, циклическим (циклом) называется такой алгоритм, в котором задана некоторая последовательность действий для многократного исполнения. Эта последовательность действий называется телом цикла. Заметим, что тело цикла

указано в алгоритме один раз, но исполняться оно может многократно. Однократное исполнение тела цикла называется итерацией. Выражение, определяющее будет ли продолжаться исполнение очередной итерации или цикл завершиться, называется условием цикла. В зависимости от того, как интерпретируется условие цикла, оно может быть условием продолжения либо условием окончания цикла.

Рассмотрим сказку о репке, «блок-схема» сюжета которой приведена на рисунке 45. Действия «привлечение очередного участника сказки» и «тянут-потянут» в сказке о репке – это тело цикла. Условием является выражение «Вытянули репку». Тело цикла, кстати, в соответствии с первоисточником, было исполнено 6 раз (дед, бабка, внучка, жучка, кошка, мышка), то есть было совершено 6 итераций.

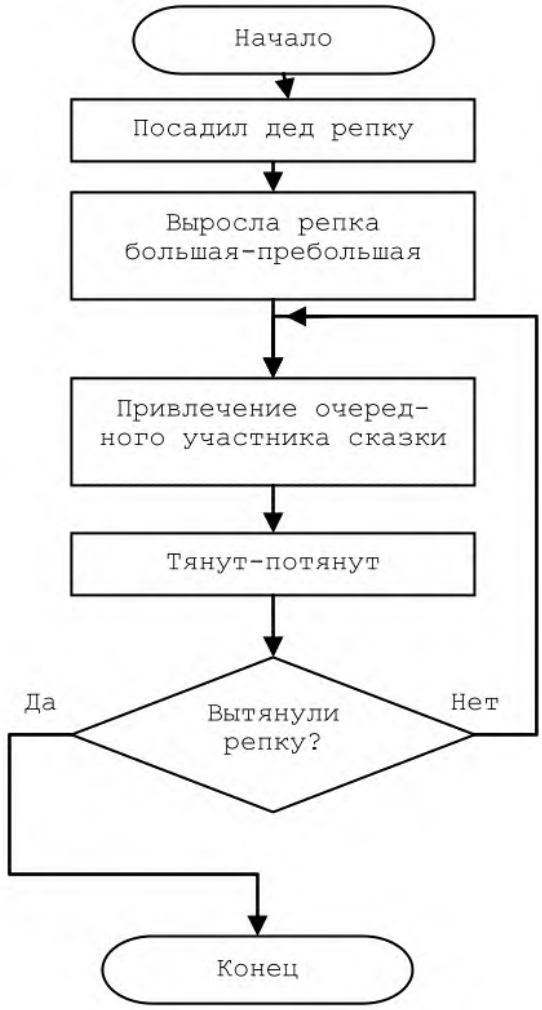


Рисунок 10. Блок-схема алгоритма сказки о репке

Существуют несколько разных видов циклических алгоритмов, отличающихся друг от друга расположением условия относительно тела, способом выхода из цикла, наличием счетчика итераций и др. Рассмотрим здесь три часто используемые разновидности циклов: цикл со счетчиком; цикл с предусловием; цикл с постусловием.

Эта разновидность цикла содержит специальную переменную (счетчик, параметр цикла), которая последовательно изменяет свое значение от заданного

начального до конечного значения с некоторым шагом, при этом для каждого значения счетчика тело цикла исполняется один раз (то есть происходит итерация). Обычно принято, что сам счетчик во время отдельной итерации не изменяется.

Так как начальное, конечное значения и шаг цикла явно заданы, то можно непосредственно до исполнения цикла заранее подсчитать количество итераций. Например, если известно, что студент вуза учится с 1 по 4 курс, причем на каждый курс он затрачивает один год, то всего он будет учиться $4 - 1 + 1 = 4$ года. Так как количество повторов исполнения тела цикла вычислимо еще до самого процесса, то цикл со счетчиком часто называют циклом с заранее известным числом итераций.

Введем условное обозначение цикла со счетчиком в виде блок-схемы (рис. 46). Здесь i – счетчик (параметр) цикла, B и E – начальное и конечное значения счетчика, S – шаг. Чаще всего шаг равен 1, в таком случае он не указывается.

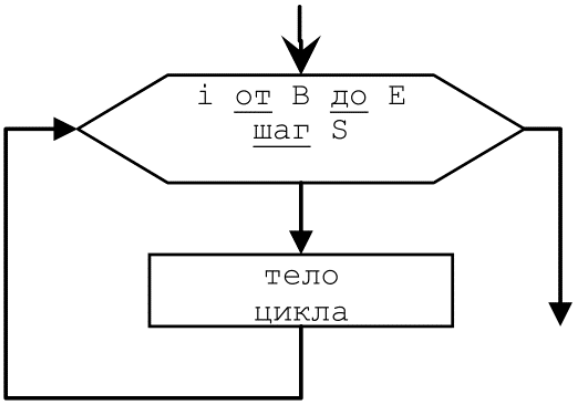


Рисунок 11. Блок-схема алгоритма цикла со счетчиком

Процесс исполнения такого алгоритма следующий. Вначале счетчик i принимает начальное значение B и исполняется тело цикла (первая итерация). Затем счетчик изменяется на шаг S, происходит вторая итерация и так далее. Когда счетчик достигнет значения E, будет исполнена последняя итерация и цикл закончится. Таким образом, количество итераций для целочисленного счетчика с шагом $S = 1$ может быть вычислено по формуле

$$C_1 = E - B + 1, B < E,$$

а с шагом $S = -1$

$$C_2 = B - E + 1, B \geq E.$$

Составить блок-схему решения следующей задачи. Дано натуральное значение n. Найти сумму всех натуральных чисел от 1 до n.

В задаче требуется по заданному натуральному n найти значение $1 + 2 + 3 + \cdots + n$.

Как бы мы поступили, если бы у нас в руках был калькулятор и нужно подсчитать, к примеру, сумму $1 + 2 + 3 + 4 + 5$? Наверное, так: сбрасываем до 0 (на экране 0); добавляем 1 (+1) – получили на экране 1; добавляем 2 (+2,

получили на экране 3); добавляем 3 (+3, получаем 6); потом +4 (10) и +5 (в итоге 15). А теперь эти же действия представим блок-схемой алгоритма.

Заметим, что в процессе вычисления у нас возникло две последовательности чисел. Первая $-1, 2, 3, 4, 5$; это слагаемые искомой суммы. Вторая – $0, 1, 3, 6, 10, 15$, это промежуточные суммы, они получаются последовательным прибавлением слагаемых; последняя сумма 15 есть ответ. В алгоритме (см. рис. 47) значения слагаемых последовательно принимает переменная i (счетчик цикла), значения сумм - переменная S.

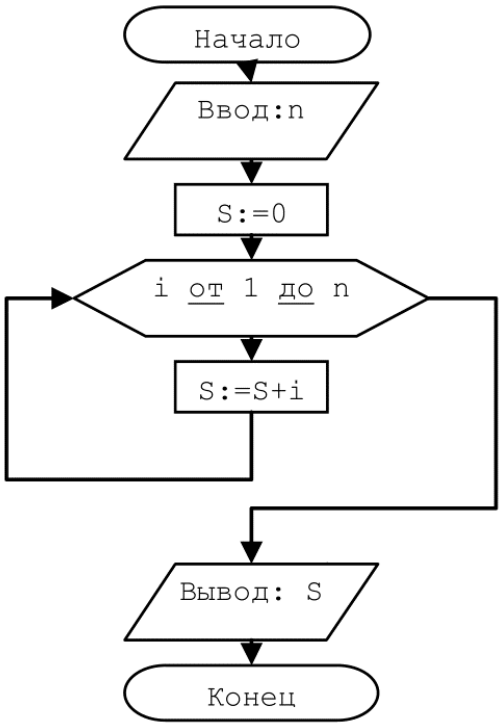


Рисунок 12. Блок-схема алгоритма расчета суммы всех натуральных чисел от 1 до n

В этом алгоритме был использован прием накопления суммы, который заключается в последовательном прибавлении к переменной суммирования S очередных слагаемых. Предварительно S инициализируется нулем.

Составить блок-схему решения следующей задачи. Дано натуральное значение n. Найти $n!$.

Значение факториала числа п равно $n! = 1 \cdot 2 \cdot 3 \cdot \ldots \cdot n$. Значит, здесь нужно найти произведение всех натуральных чисел от 1 до n. Поступим здесь аналогично предыдущему примеру– составим алгоритм с накоплением произведения (рис. 48). Переменная, в которой будем накапливать произведение, инициализируется единицей, а затем в цикле она домножается на очередной множитель.

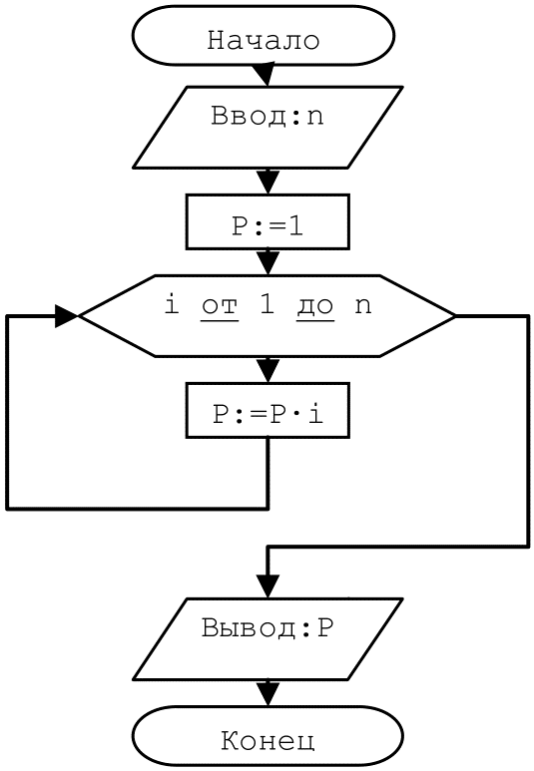


Рисунок 13. Блок-схема алгоритма расчета $n!$

Составить блок-схему решения следующей задачи. Дано натуральное значение n. Найти $\sum_{i=1}^{n} i!$.

В данной задаче нужно найти значение $1! + 2! + \cdots + n!$. Например, для $n = 3$ это значение равно $1! + 2! + 3! = 1 + 1 \cdot \ \ 2 + 1 \cdot 2 \cdot 3 = 1 + 2 + 6 = 9$.

С одной стороны, здесь необходимо найти сумму - значит, нужен алгоритм с накоплением суммы. С другой стороны каждое слагаемое – это факториал, то есть произведение. Вычислять каждый факториал накоплением слишком дорого, так как нужно начинать все время с единицы и повторять много уже совершенных действий. Воспользуемся следующим соотношением, а именно – факториал числа i равен факториалу $i-1$, домноженному на i : $i! = (i-1)!\, i$ (это верно для $i \geq 1$). Поэтому параллельно накоплению суммы будем накапливать факториал – произведение натуральных чисел. Алгоритм решения для $n = 3$ приведены на рис. 49. Заметим, что в блок-схеме к переменной S добавляются слагаемые P, так как здесь суммируются факториалы.

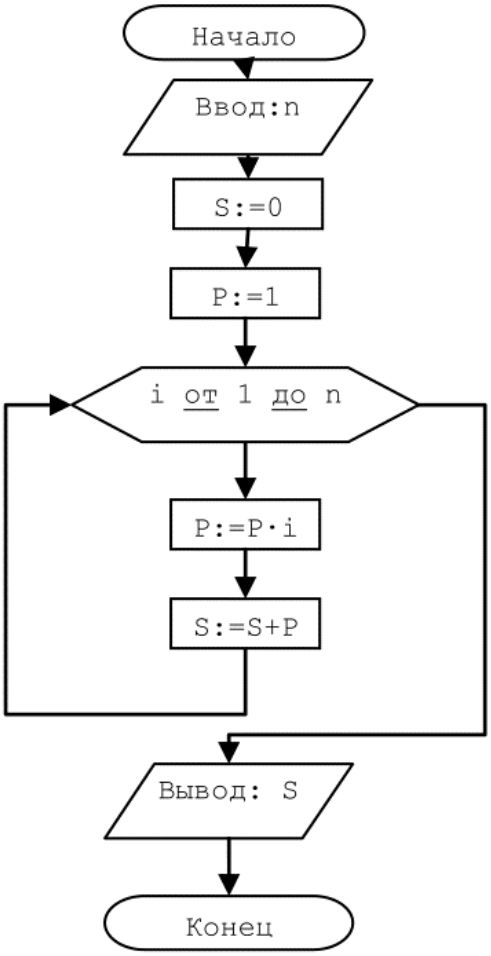


Рисунок 14. Блок-схема алгоритма расчета $\sum_{i=1}^{n} i!$

Цикл с предусловием – это цикл, исполнение которого продолжается, пока истинно условие цикла, проверяемое до исполнения очередной итерации. Так как при истинности условия цикл продолжается, то условие называют «условием продолжения цикла».

Если окажется, что с самого начала условие цикла ложно (false, F), то тело цикла ни разу не будет исполнено (0 итераций). Если в процессе исполнения цикла условие всегда принимает значение «истина» (true, T), то цикл начинает исполняться бесконечно – происходит зацикливание. Эта ситуация ошибочна, поэтому необходимо, чтобы условие непременно в какой-то момент стало бы ложным. Тогда произойдет корректный выход из цикла.

Составить блок-схему решения следующей задачи. Даны действительные числа a и b $(a > 1)$. Найти все члены последовательности $a_1, a_2, a_3, \ldots$, меньшие b.

Заметим, что количество выходных значений в этой задаче может быть различным – это зависит от входных данных. В том числе возможна ситуация, когда на выходе нет ни одного числа (рис. 50).

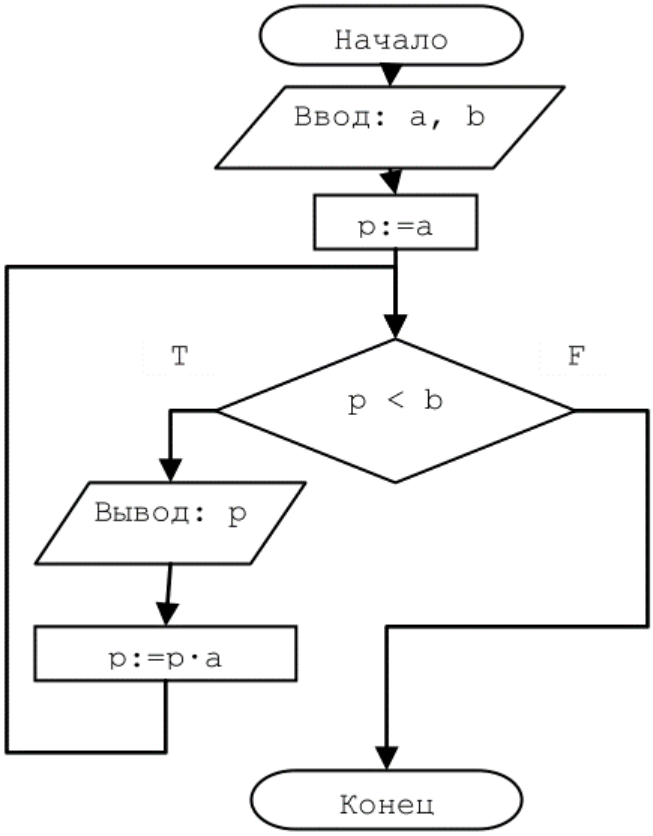


Рисунок 15. Блок-схема алгоритма нахождения чисел, меньше заданного

Цикл с постусловием – это цикл, исполнение которого продолжается, пока ложно условие цикла, проверяемое после исполнения тела цикла.

Так как при истинности условия цикл прекращается, то постусловие называют «условием окончания цикла».

Из блок-схемы видно, что независимо от начального значения постусловия тело цикла будет исполнено по крайней мере один раз. Также здесь возможна ситуация зацикливания, когда при исполнении постусловие всегда принимает значение «ложь» (F).

Заметим, что рассмотренный ранее алгоритм сказки о репке содержит цикл с постусловием.

Составить блок-схему решения следующей задачи. Дано натуральное число N. Найти первое из чисел Фибоначчи, большее N.

Построим алгоритм, исходя из следующих соображений. Будем последовательно получать числа Фибоначчи. Каждое вновь полученное число будем сравнивать с N, и, если оно окажется больше N, процесс следует остановить. Так как в этом алгоритме вначале получаем число, а только потом проверяем, не является ли оно искомым, то разумно использовать цикл с постусловием, когда проверка осуществляется последействия. Блок-схема алгоритма приведена на рисунке 51.

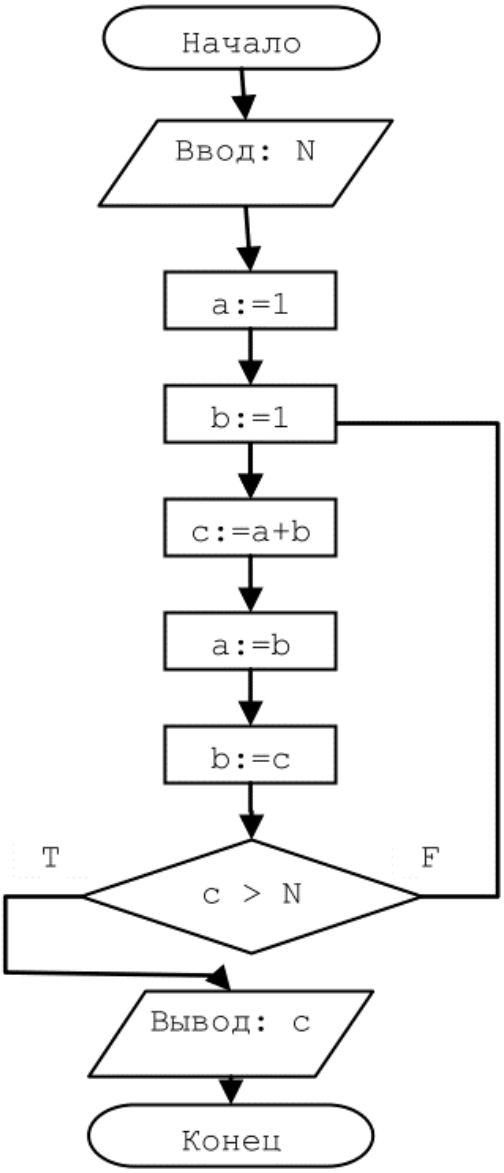


Рисунок 16. Блок-схема алгоритма нахождения первого числа Фибоначчи, больше заданного

Все циклические алгоритмы можно разделить на две группы: циклы с заранее известным количеством повторений и циклы с заранее неизвестным количеством повторений [29]. Циклы с заранее неизвестным количеством повторений называются итерационными. Итерационный процесс – это последовательное приближение к результату за некоторое количество шагов.

Рассмотрим блок-схемы итерационных циклов на примере.

Даны действительные числа x, ε $(x \neq 0, \varepsilon > 0)$. Вычислить сумму с точностью ε: $\sum_{k=0}^{\infty} \frac{x^k}{2^k k!}$.

Вычисление бесконечной суммы с заданной точностью ε означает, что требуемая точность достигнута, когда вычислена сумма нескольких первых слагаемых и очередное слагаемое оказалось по модулю меньше, чем ε, – это и все последующие слагаемые можно не учитывать.

Пусть c – слагаемое, S – сумма, F – факториал числа k.

Для вычисления факториала числа можно воспользоваться формулой $F = F \cdot k$. Начальное значение $F = 1$.

Выведем общую формулу для вычисления одного слагаемого (без учета факториала числа k):

$$k = 0, c = \frac{x^0}{2^0} = 1,$$

$$k = 1, c = \frac{x^1}{2^1} = \frac{x}{2},$$

$$k = 2, c = \frac{x^2}{2^2} = \frac{x}{2} \cdot \frac{x}{2} = c \cdot \frac{x}{2},$$

$$k = 3, c = \frac{x^3}{2^3} = \frac{x^2}{4} \cdot \frac{x}{2} = c \cdot \frac{x}{2},$$

$$k = 4, c = \frac{x^4}{2^4} = \frac{x^3}{8} \cdot \frac{x}{2} = c \cdot \frac{x}{2}.$$

Обобщая, для произвольного значения k можно записать $c = c \cdot \frac{x}{2}$, где значение переменной c в правой части формулы вычисляется на предыдущем шаге. Начальное значение $c = 1$ при $k = 0$.

Сумму вычисляем по формуле $S = S + c/F$. Начальное значение $S = 0$.

В данной задаче следует использовать цикл с предусловием, потому что количество слагаемых будет зависеть от введенного значения x и требуемой точности вычислений ε.

Переменная k – целого типа, переменные x, ε, c, S – вещественного типа. Переменную F следует взять вещественного типа, так как диапазон целых чисел ограничен, а факториал быстро возрастает с ростом значения k.

Приведем словесное описание и блок-схему алгоритма.

1. Ввод исходных данных x и ε.

2. Присваивание начальных значений $F = 1, S = 0, c = 1$. Параметр цикла $k = 1$.

3. Прежде чем вычислять сумму, определим выполнение условия продолжения цикла. По условию, когда очередное слагаемое окажется по модулю меньше заданной точности ε, вычисления следует прекратить. Слагаемым в нашем случае является дробь c/F. Так как может быть введено отрицательное значение x, то использована функция abs(), определяющая модуль числа.

4. Если требуемая точность вычислений не достигнута (выход «Да» блока 3), выполняем тело цикла, определяем новые значения переменных и продолжаем цикл.

5. Как только требуемая точность вычисления достигнута (выход «Нет» блока 3), завершаем цикл и печатаем значение переменной S.

Цикл, который мы использовали при решении задачи, называется итерационным (рис. 52). Особенностью такого цикла является то, что число его повторений зависит от выполнения условия, записанного при входе в цикл. В итерационных алгоритмах необходимо обеспечить обязательное достижение условия выхода из цикла (сходимость итерационного процесса). В противном случае произойдет «зацикливание» алгоритма. Управление итерационным циклом организует программист, который должен позаботиться и об инициализации управляющей переменной, и об ее приращении, и об условии завершения цикла.

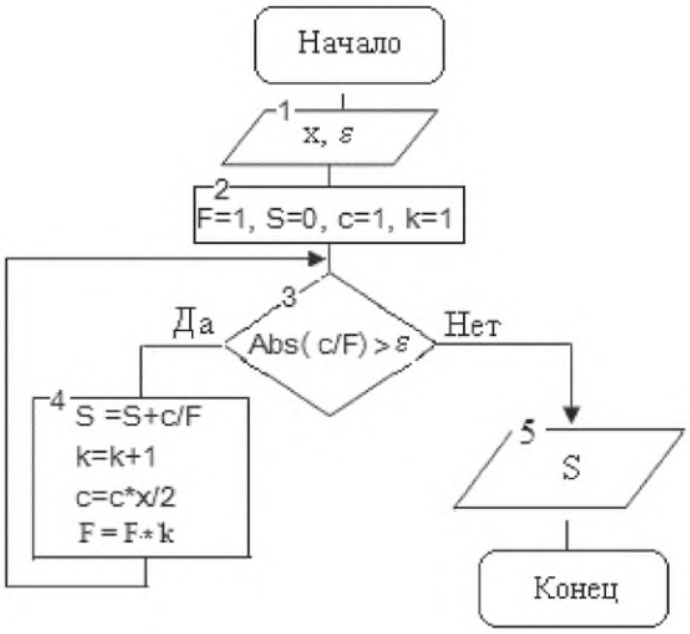


Рисунок 17. Блок-схема алгоритма расчета бесконечной суммы

1.3. Реализация нелинейной структуры алгоритма

Поток управления – это последовательность передач управления между операторами [27]. Он задает порядок выполнения операторов.

Программа вводится в компьютер как последовательность строк. Отдельные строки программы могут выполняться неоднократно, приводят к долгим вычислениям. Например, статическая строка

```
for i from 1 to 3 do writein (i, i * i );
```

вызывает динамические вычисления:

```
writeln (1, 1 • 1)
writeln (2, 2 x 2)
writeln (3, 3*3)
```

Статический текст программы может значительно отличаться от динамических вычислений, выполняемых при работе программы. Это затрудняет понимание программы, увеличивает возможность внесения ошибки.

Для облегчения понимания операторы императивного языка программирования должны обеспечивать структурированный поток управления.

Структурированным называют поток управления, который удобен для понимания человеком и максимально близок к структуре текста программы.

Программа является структурированной, если ее поток управления очевиден из структуры программного текста.

Многочисленные исследования показали, что структурированность обеспечивают языки, операторы которых имеют по одному входу и одному выходу. Структурированный поток управления может принимать линейную, разветвляющуюся и циклическую формы. Считается, что эти формы позволяют человеку легко отслеживать логику вычислений в программе.

Средствами обеспечения структурирования являются три структуры управления: композиция, ветвление, цикл:

1. Композиция. Эта структура является средством группировки линейной последовательности простых операторов.

2. Ветвление. Структура обеспечивает выбор для выполнения одной из нескольких альтернативных последовательностей операторов.

3. Цикл. Последовательность операторов может повторяться многократно, ноль или более раз. Ноль означает, что последовательность может быть вообще пропущена.

Эти структуры рассматриваются как строительные блоки, из которых, многократно комбинируя их, мы можем создать сколь угодно большие структурированные программы. Процесс же конструирования программ из этих блоков называют структурированным (а иногда – структурным) программированием.

Здесь возникает законный вопрос: не приведет ли использование только таких структур управления к ограничению возможностей программ и программирования как такового? Ответ давался многими исследователями, утверждавшими: ограничения на общность и универсальность решений не возникают! Среди наиболее ярких результатов отметим структурную теорему Бема-Джакопини и теорему о регуляризации алгоритмов выдающегося отечественного ученого В. М. Глушкова.

Впрочем, надо иметь и виду следующее: фактически доказано лишь то. что любую программу можно написать, используя только перечисленные структуры управления. Получаемый результат вовсе не обеспечивает автоматически лучшее решение. Известно, что существует некоторое решение не хуже (а может, и лучше) структурированного результата.

В языках программирования описанные структуры управления реализуются специальным набором операторов управления, которые представляют собой операторы с одной точкой входа и одной точкой выхода. Каждый оператор

управления может содержать внутренние ветвления и циклы, но передача управления из него к другому оператору происходит только через его единственную точку выхода. Все эти рассуждения ориентированы на поддержку следующего утверждения: порядок выполнения операторов программы должен соответствовать порядку операторов в ее тексте.

Составной оператор представляет собой последовательность операторов, которую при создании более сложных операторов можно рассматривать как единый оператор. Впервые составной оператор появился в языке Algol 60. Обычно составные операторы записывают в виде:

```
begin
...      - последовательность из нескольких операторов
end
```

Здесь несколько операторов помещаются между открывающей begin и закрывающей end скобками. Пример составного оператора: begin

```
temp := x;
x y;
y := temp end
```

В Паскале между операторами последовательности ставят точки с запятой. Лучший вариант – точки с запятой должны завершать операторы, то есть появляться после каждого оператора.

В языках C, C++, C#, Perl и Java границы составного оператора обозначаются открывающей и закрывающей фигурной скобкой: В некоторых языках (C, Ada) в начало составного оператора можно добавлять объявления данных, превращая его в блок.

Операторы, входящие в составной оператор, записываются в той последовательности, в которой они должны выполняться. Таким образом, составной оператор группирует линейный поток управления и реализует структуру управления «композиция».

Поскольку составной оператор – это отдельный оператор, то операторы некоторого фрагмента вычисления можно объединять вместе, применяя скобки begin... end, причем можно составлять иерархические структуры из таких фрагментов.

Язык Pascal унаследовал от языка Algol способ формирования составных операторов, но не позволяет создавать блоки. Некоторые языки не нуждаются в специальном разграничении составных операторов, интегрируя их в свои структуры управления.

Условный оператор обеспечивает возможность альтернативного выполнения одной из двух последовательностей (ветвление) или выполнение какой-либо одной

последовательности при определенных условиях. Термин последовательность здесь обозначает либо одиночный элементарный оператор, либо составной оператор, либо другой оператор управления. Выбор осуществляется путем проверки выражения ветвления, результат которой имеет истинное или ложное значение.

В самом общем виде можно записать условный оператор с двумя ветвями:

```
if выражение_ветвления
then последовательность_1
else последовательноеть_2;
```

В зависимости от вычисленного значения выражения ветвления (истина или ложь), выполняется соответственно либо последовательность_1 операторов, либо последовательность_2 операторов.

Условный оператор с одной ветвью

```
if выражение_ветвления then
```

последовательность; обеспечивает выполнение последовательности при значении условия истина и пропуск последовательности при значении ложь.

Приведем пример условного оператора на языке Pascal:

```
if x > 0
then y := 1/x
else
begin
x 2; y 1/z
end
```

Последовательность в ветви then здесь является простым оператором присваивания. а последовательность в ветви else – составным оператором. Заметим, что точку с запятой в конце then-последовательности ставить нельзя.

Перепишем этот оператор для языка С:

```
if (x > 9) y - 1/x;
else {
x = 2;
y = 1/z;
}
```

В этом языке (и языках С++, С#,.Java) нет ключевого слова then, а проверяемые выражения в условных операторах и операторах цикла должны заключаться в круглые скобки. Выражение ветвления здесь считается арифметическим

выражением. которое возвращает значение 0 (ноль моделирует ложь) или 1 (единица моделирует истину). В языках C++. C#, Java для этих целей используют логические значения. Наконец, точку с запятой в конце then-последовательности здесь надо ставить обязательно.

Арифметическое выражение в ветвлении используется также в языке Python, а если быть точными, то в C++ может применяться как арифметическое, так и логическое выражение.

Во многих современных языках ветви then и else могут содержать как единичные, так н составные операторы. Исключением является язык Perl, в котором обе ветви должны быть составными операторами, даже если они содержат единичные операторы. Многие языки для формирования составных операторов используют скобки, которые ограничивают «тела» ветвей then и else. В языках Fortran 95, Ada, Python и Ruby ветви then и else являются последовательностями операторов, а не составными операторами. Весь условный оператор завершается в этих языках зарезервированным словом или фразой (например, в языке Ada применяется end if).

Язык Python для спецификации составных операторов использует отступы. Приведем пример:

```
if a > b :
a = b
print «первый вариант»
```

Все операторы с одинаковым отступом включены в составной оператор. Оператор, следующий за составным оператором, должен иметь такой же отступ, что и предложение if. Заметим, что вместо then здесь используется двоеточие.

В функциональных языках ML. F# и LISP разветвление не является оператором: это выражение, которое возвращает значение. Следовательно, оно может появиться там, где уместны любые другие выражения. Рассмотрим следующий пример на языке F#:

```
let m =
if a > 0 then a
else 5 * a;
```

Здесь создается имя т, которое инициируется величиной а или выражением 5 * а, в зависимости от результатов анализа значения а.

Вариации формы ветвей влияют на смысл вложенных условных операторов.

Интересная проблема появляется, если допускается вложение условных операторов. Рассмотрим следующий условный оператор на языке Pascal:

```
if x1 > y1 then
```

```
if x2 > y2 then
оператор_1
else
оператор_2;
```

Здесь возникает вопрос – какому из двух if соответствует else? Отступы в написанном тексте намекают на желание программиста связать else с первым if. Но это лишь желание... Увы. синтаксис не даст указаний о соответствии. За исключением языков Python и F#. отступы не влияют на семантику и игнорируются компиляторами. Суть проблемы состоит в том. что ветвь else следует за двумя ветвями then без промежуточной ветви else, и нет никаких синтаксических индикаторов, указывающих на соответствие ветви else одной из ветвей then.

Для предотвращения неоднозначности в языках Pascal, Java, как и во многих других императивных языках, придумывается дополнительное, внутреннее правило. связывающее else с ближайшим предшествующим if и. следовательно, со вторым then. Говорят, что ветвь else всегда образует пару с ближайшей предшествующей ветвью then, не имеющей нары. Следовательно, в этом примере ветвь else должна составить пару второй ветви then. Недостаток использования такого правила состоит в том. что. хотя программист и может считать else альтернативой первому then, компилятор на уровне синтаксиса его ошибку не выявит, но будет считать семантику прямо противоположной. Чтобы ввести (например, на языке Java) альтернативную семантику, нужно внутренний if сделать составным оператором:

```
if (xl > yl) {
if (x2 > y2)
оператор_1;
}
else
оператор_2;
```

В языках C, C++ и C# существует та же проблема вложенности условного оператора. Поскольку язык Perl требует, чтобы все ветви then и else были составными, здесь проблемы вложенности нет. На языке Perl предыдущий пример записывается в следующем виде:

```
if (x1 > y1) {
if () {
оператор_1;
}
```

```
} else {
оператор 2;
)
```

Если же требуется альтернативная семантика, то надо записать:

```
if (x1 > y1) {
if (x2 > y2) {
оператор_1;
}
else {
оператор_2;
}
}
```

Оператор выбора позволяет произвести ветвление по одному из многих вариантов. Причем каждый вариант может включать один оператор или целую группу операторов. Следовательно, оператор выбора может рассматриваться как обобщение условного оператора. Конечно, ветвление по многим направлениям можно реализовать и с помощью условных операторов, но такое решение оказывается трудным для чтения, громоздким и ненадежным.

Оператор выбора в языке Rascal называется case-оператором и имеет следующий вид:

```
case выражение of
список констант 1: оператор_1;
список констант n: оператор_n;
end
```

Количество значений выражения должно быть счетным (дискретным). Выбираемые операторы могут быть либо отдельными, либо составными. Семантика оператора заключается в следующем: вычисляется выражение, и его значение сравнивается с константами во всех списках констант. Если найдено совпадение, то управление передается оператору, приписанному к соответствующей константе. После выполнения этого оператора происходит выход из case-оператора.

Константы в разных списках должны быть различными, то есть константы не могут появляться в нескольких списках констант. Требование о покрытии константами всех значений выражения не выдвигается. Эта проблема была просто проигнорирована. Во многих современных версиях этого языка в case-операторvдобавляется ветвь else, которая выполняется, если значение выражения не при надлежит ни одному из списков констант.

Формы оператора выбора в различных языках различны, однако все они таран тируют следующее:

- константы вариантов могут появляться в любом порядке:
- с помощью различных констант может выбираться один и тот же вариант;
- константы разных вариантов должны быть различны.

Повторение некоторой последовательности операторов – это мощное средство для сокращения длины программ и повышения эффективности вычислений. Отсутствие возможности повторения группы операторов привело бы к существенному усложнению программ и резкому возрастанию времени на программирование.

Известны две схемы организации повторений: рекурсия и итерация.

Рекурсия – это такой способ организации повторений, при котором последовательность операторов вызывает сама себя непосредственно либо с помощью других последовательностей. Она является естественной математической формой записи повторяющихся вычислений (на основе функций). Рекурсия считается главным средством поддержки повторений в функциональных языках, которые рассматриваются в главе 20.

Итерация – это способ многократного повторения последовательности операторов, реализуемый в виде цикла. В языках программирования циклы поддерживаются итеративными конструкциями, называемыми операторами циклов.

В общем случае термин «итерация» имеет два значения: 1. процесс выполнения последовательности повторяемых шагов; 2. отдельный шаг из последовательности повторяемых шагов. Опираясь на второе значение, говорят об «отдельной итерации цикла».

Оператор цикла обеспечивает многократное выполнение встроенных в него операторов, являясь, по сути, структурой управления. В состав оператора цикла входят секция управления и тело.

Тело цикла образует произвольный набор операторов, выполнением которых руководит оператор цикла. Этот набор определяет функциональные возможности цикла. Секция управления управляет количеством повторений тела.

К настоящему времени разработано несколько разновидностей операторов цикла. Их специфика определяется механизмом работы секции управления и ее размещением относительно тела цикла.

Секция управления может быть размещена как перед телом, так и после тела цикла.

В основу механизма работы секции управления может быть положен анализ логического выражения или подсчет числа повторений. Иногда эти две возможности комбинируют. В результате формируйся условие продолжения, или условие завершения цикла.

Кроме того, различают операторы цикла с предусловием (проверка условия предваряет выполнение тела) и с постусловием (проверка условия проводится после выполнения тела).

Операторы цикла с заданным числом повторений задают заранее известное количество повторений тела. В составе их секций управления имеется так называемая переменная цикла. Переменная цикла принимает дискретное количество значений и используется для подсчета количества повторений тела, поэтому иногда ее называют счетчиком. Секция управления ограничивает количество значений переменной цикла, определяя ее начальное, конечное значение, а также величину шага между соседними значениями. Начальное и конечное значения переменной, а также величину шага называют параметрами цикла.

Наиболее простой вариант подобного оператора цикла предлагается в языке Pascal. Он записывается в следующем виде:

for переменная := начальное_значение to конечное_значение do оператор_тела

Секцией управления здесь является конструкция:

for переменная := начальное_значение to конечное_значение

Опишем семантику данного for-оператора. Сначала переменная (счетчик) принимает начальное значение и выполняется тело, затем переменная увеличивается па единицу и тело снова выполняется и гак далее, пока значение переменной не превысит конечного значения. В момент превышения цикл прекращается. Заметим, что оператору тела (составному или простому) запрещено изменять значение переменной цикла. Считается, что при выходе из цикла значение переменной цикла не определено.

Для этого оператора обеспечивается и другой вариант: уменьшение переменной цикла на единицу перед каждом повтором. Достаточно лишь применить секцию управления с иным ключевым словом:

for переменная := начальное_значение downto конечное_значение

При этом условие прекращения цикла приобретает вид:

переменная < конечное_значение

Отметим также два обстоятельства. Во-первых, условие, управляющее работой for-оператора, проверяется перед выполнением тела. Если условие не выполняется в самом начале работы оператора, тело не будет выполнено ни разу. Во-вторых, величина шага не меняется (является постоянной).

Похожие операторы встречаются и в других языках программирования. Специфику их реализации обычно определяют следующие решения:

- переменная цикла задастся внутри оператора цикла или вне его;

- параметры цикла вычисляются однократно или многократно.

Однократное вычисление параметров цикла гарантирует простоту реализации оператора, в то время как многократное вычисление существенно увеличивает гибкость оператора.

Операторы цикла без заданного числа повторений количество повторений тела заранее не известно. Решение о прекращении цикла принимается по результатам анализа текущей ситуации. В этом случае говорят о логически управляемых циклах. Подобные циклы соответствуют итерациям более общего вида: любой цикл с управлением по счетчику можно построить с помощью логически управляемого цикла, но не наоборот.

Иногда условия выхода из цикла неизвестны и не могут быть выражены в секциях управления операторов цикла. В таких ситуациях языки предлагают конструкции бесконечных циклов. Например, в языке Ada бесконечный цикл имеет вид:

```
loop
операторы тела
end loop;
```

В языке Pascal бесконечный цикл организуется с помощью оператора while с условием, которое всегда выполняется:

```
while true do begin операторы тела end
```

Аналогичную роль играет следующая конструкция языка C:

```
for(; ;){ операторы тела }
```

В подобных случаях программисту приходится встраивать в тело свой собственный механизм управления циклом.

В циклах с предусловием секция управления размещается перед телом. Наиболее популярной конструкцией цикла с предусловием считается оператор while (язык Pascal):

```
while условие продолжения do тело
```

Тело повторяется, пока условие имеет истинное значение.

В языке Ada оператор цикла while имеет следующий вид:

```
while условие продолжения loop
последовательность операторов
end loop;
```

Порядок выполнения оператора:

1. Вычисляется значение условия. Если значение равно True, выполняется переход к пункту 2. В противном случае (при значении False) происходит выход из цикла.

2. Выполняются операторы тела цикла. Осуществляется переход к пункту 1.

Таким образом, это цикл с условием продолжения, предваряющим исполнение тела.

Отметим характерные особенности оператора while:

1. Операторы тела могут выполняться пуль и более раз.

2. Операторы тела повторяются пока условие равно True.

3. В теле должен быть оператор, влияющий па значение условия (для исключения бесконечного повторения).

В циклах с постусловием секция управления размещается после тела. Можно вообразить ситуацию, когда проверку условия продолжения следует проводить после выполнения тела цикла. В таких ситуациях языки C, C++, C# и Java рекомендуют оператор do-while:

```
do
тело цикла
while (условие продолжения)
```

Тело цикла с последующей проверкой условия повторяется до тех пор, пока условие не станет ложным (нулем). Единственное различие между операторами do -while и while состоит в том. что оператор do-while всегда заставляет тело цикла выполняться хотя бы один раз. В обоих случаях оператор тела является составным.

При применении оператора do-while предыдущий пример надо переписать в следующей форме:

```
do {
i = i - 1;
y += a;
} while (i != c)
```

Если условие выполняется, управление пере дастся на другую итерацию цикла.

Оператор цикла с постусловием repeat-until языка Pascal имеет следующий синтаксис:

```
repeat
тело цикла
until условие окончания
```

Тело цикла повторяется до тех пор. пока условие окончания не установится в значение True. Таким образом, логика управления оператора repeat-until противоположна логике управления оператора do-while языков C, C++, C# и Java.

Проиллюстрируем сказанное, переписав предыдущий пример в терминах оператора repeat-until:

```
repeat
i := i - 1;
y := y + a
until i = c
```

Если условие выполняется, происходит выход из цикла.

В общем случае цикл с постусловием считают менее полезным и лаже опасным, поскольку программисты часто забывают о том, что его тело всегда должно выполняться хотя бы один раз.

В основе *структур управления ветвлением и циклами*, созданных Э. Дейкстрой, лежало стремление гарантировать правильность потока управления.

Он предложил следующую структуру охраняемого ветвления:

```
if предохранитель -> последовательность_операторов
[ ] предохранитель -> последовательность_операторов
[ ] -
[ ] предохранитель -> последовательность_операторов
fi
```

Замыкающая скобка fi представляет собой открывающую скобку if, записанную наоборот. Маленькие блоки [ ]. названные барьерами, используются для отделения охраняемых операторов. Каждая строка в структуре называется охраняемой командой.

Эта структура ветвления имеет следующую семантику. Все предохранители (булевы выражения) вычисляются каждый раз при достижении этой структуры в программе. Если истинными являются несколько предохранителей, то для выполнения выбирается (случайным образом) одна из последовательностей операторов. Если ни один предохранитель nt имеет истинного значения, вычисления прекращаются. Такой подход вынуждает программиста рассматривать все возможности ветвления.

Охраняемые команды Дейкстры являются наглядной иллюстрацией влияния синтаксиса и семантики операторов на проверку правильности программ (и обратного влияния). Многочисленные исследования доказали, что проверка правильности программ очень затруднительна, если используются операторы безусловного перехода. Проверка существенно упрощается, если используются

только классические структурированные средства управления либо только охраняемые команды. Однако, к сожалению, сложность реализации охраняемых команд очень высока.

Инварианты являются ключевым понятием при разработке императивных программ.

Инвариантом в некоторой точке программы является утверждение, которое сохраняет свой смысл при достижении этой точки во время выполнения программы, то есть при достижении управлением этой точки. По сути, утверждение – это условие, которое в зависимости от состояния вычислений принимает значение истина или ложь. Пример – условие x > y, которое относится к значениям величин x и y.

Главная трудность в написании корректной программы состоит в том. что правильность – это свойство не ее статического текста, а свойство динамических вычислений, проводимых программой в процессе работы.

Инварианты помогают судить о вычислениях программы, играют роль моста между статическим программным текстом и динамическим процессом вычислений.

Когда мы говорим об инвариантах, на самом деле мы работаем с утверждениями, поскольку имеем дело со свойствами, которые сохраняются при достижении управлением конкретной программной точки.

Плохой способ – написать программу, а затем модифицировать инварианты. Лучший способ – начать с инвариантов и использовать их для разработки программы.

Например. разработка программы, устраняющей смежные дублирования, может стартовать с инварианта:

```
{здесь: x - первый элемент данных из анализируемой серии}
```

Продолжим создание программы. Выведем элемент на экран, а затем многократно читаем элементы, пропуская дубликаты x. Читаемые элементы заносим в переменную next до тех пор. пока значение next не станет отлично от значения x:

```
writein (x);
repeat read (next) until next <> x;
{здесь: несколько раз прочитали элементы с одним и тем же значением}
```

Элементе новым значением заносим в x:

```
x := next;
{здесь: организуем новую серию действий по устранению дубликатов}
```

Конечная версия программы примет вид:

```
read (x);
while x <> маркер конца списка do begin
```

```
{здесь: x - первый элемент данных из анализируемой серии)
writein (x);
repeat read (next) until next <> x;
{здесь: несколько раз прочитали элементы с одним и тем же значением)
x := next;
{здесь: организуем новую серию действий по устранению
    дубликатов}
end
```

При использовании конструкций типа «один вход/один выход» поток управления в программе очевиден из ее текста.

Как это может помочь разработке программы?

Если оператор имеет один вход и один выход, то его поведение может быть задано двумя условиями:

1. условием, выполняемым перед входом в оператор (предусловием).
2. условием, выполняемым после выхода из оператора (постусловием).

Иными словами, поведение оператора определяется инвариантными утверждениями.

Например: для цикла while эти утверждения должны формулировать:

- предусловие – условия, требуемые для выполнения цикла;
- инвариант внутри тела цикла – условия, требуемые для нахождения в цикле;
- постусловие – условия, справедливые по выходу из цикла.

Рассмотрим оператор:

```
while x > y do
x := x - y;
```

Очевидно, что каждый раз перед присваиванием x : = x - y должно выполняться x > y.

Запишем это инвариантное утверждение:

```
while x > y do
{x > y, если мы здесь)
x := x - y;
```

Предположим, что перед входом в цикл справедлив инвариант (предусловие):

```
{x > 0 and y > 0}
```

Проверка выражения x > y между while и do будет гарантировать, что x > y перед присваиванием x : = x - y.

```
{x > 0 and y > 0}
```

```
while x > y do
{y > 0 and x & y}
x := x - y;
```

После присваивания измененное значение x должно удовлетворять условию x > 0.

Положим, что старое значение x равно a. Из y > 0 и a > y следует, что a - y > 0. Поскольку измененное значение x = a - y (после присваивания), мы получаем x > 0.

Таким образом, записываем:

```
{x & 0 and y > 0}
while x > y do
{y > 0 and x & y}
x := x - y;
(x > 0 and y > 0}
```

Управление опять передается на заголовок. После присваивания инвариант сохраняется для следующей итерации.

СПИСОК ЛИТЕРАТУРЫ

1. Игошин В.И. Математическая логика и теория алгоритмов : учеб, пособие для студ. высш. учеб, заведений / В. И. Игошин. – 2-е изд. стер. – М.: Издательский центр «Академия». 2008. – 448 с.

2. Глухов М. М., Шишков А. Б. Математическая логика. Дискретные функции. Теория алгоритмов: Учебное пособие. – СПб.: Издательство «Лань», 2012. – 416 с.

3. Геут К. Л. Математическая логика и теория алгоритмов : учсб.-метод. пособие / К. Л. Геут, С. С. Титов. – Екатеринбург : УрГУ ПС, 2017. – 85 с.

4. Мендельсон Э. Введение в математическую логику. – М.: Наука, 1976. – 320 с.

5. Зюзьков В. М. Математическая логика и теория алгоритмов : учебное пособие / В. М. Зюзьков. – Томск : Эль Контент, 2015. – 236 с.

6. Успенский В. Л. Труды но нематематике / В. А. Успенский. 2-е изд,, испр. и доп. : В 5 кн.– М. : ОГИ : Фоад «Магматические этюды», 2012. – Кн. 2 : Философия (с приложением трёх писем П. .П. Трауберг к автору). – 2014. – 566 с.

7. Лихтарников Л. М., Сукачева Т. Г. Математическая логика. Курс лекций. Задачник-практикум и решения: Учебное пособие. 4-е изд., стер. – СПб.: Издательство «Лань», 2009. – 288 с.

8. Вялый М., Подольский В., Рубцов А., Шварц Д., Шень А. Лекции по дискретной математике. 448 с. https://publications.hse.ru/mirror/pubs/share/direct/393719078.pdf

9. Пальчунов, Д. Е. Математическая логика и теория алгоритмов : учеб, пособие / Д. Е. Пальчунов ; Новосиб. гос. ун-т. - Новосибирск, 2016. – Ч. 1. – 96 с.

10. Барашев В.П.. Унучек С.А. Дискретная математика : Учебное пособие Федеральное государственное бюджетное образовательное учреждение высшего профессионального образования «Московский государственный технический университет радиотехники, электроники и автоматики». – М., 2012. –268 с., электронное издание.

11. Чашкин А. В. Дискретная математика : учебник для учреждений высш. проф. образования / А. В. Чашкин. – М.: Издательский центр «Академия», 2012. – 352 с.

12. Гринченков Д.В. Математическая логика и теория алгоритмов для программистов : учебное пособие / Д.В. Гринченков, С.И. Потоцкий. – М.: КНОРУС, 2010. – 208 с.

13. Вайнштейн К. В. Математическая логика и теория алгоритмов : учеб, пособие / Ю. В. Вайнштейн, Т. Г. Пенькова, В. И. Вайнштейн. – Красноярск : Сиб. федер. ун-т, 2019. – 110 с.

14. Новиков Ф. А. Дискретная математика для программистов. Учебник для вузов. 3-е изд. – СПб.: Питер, 2009. – 384 с.

15. Марченков С. С. Основы теории булевых функций. М.: ФИЗМАТЛИТ, 2014. – 136 с.

16. Гаврилов Г. П., Сапоженко А. А. Задачи и упражнения по дискретной математике: Учеб, пособие. – 3-е изд., перераб. – М.: ФИЗМАТЛИТ, 2005. – 416 с.

17. Кормен Т. Х., Лензерсон, Ч. И., Ривест Р. Л., Штайн К. Алгоритмы: построение и анализ. 2-е издание : Пер. с англ. – М. : Издательский дом «Вильямс», 2011. – 1296 с.

18. Алексеев В.В. Теория алгоритмов. Учебно-методическое пособие. СарФТИ НИЯУ МИФИ, 2021. – 100 с.

19. https://education.yandex.ru/handbook/algorithms

20. Ахо А. В., Хопкрофт Д. Э., Ульман Д. Д. Структуры данных и алгоритмы : Пер. с англ. : М. : Издательский дом «Вильямс», 2003. – 384 с.

21. Граничин О.Н. Рандомизированные алгоритмы в задачах обработки данных и принятия решений. Системное программирование. Санкт-Петербургский государственный университет, Т. 6, № 1. С. 141–162.

22. Регеда В. В., Регеда О. Н. Основы алгоритмизации. Методические указания по курсу «Информатика». Пенза: Издательство Пензенского государственного университета, 2008. – 56 с.

23. Туаева М. Т. Алгоритмы. Основы алгоритмизации: Учебное пособие. – Владикавказ: Владикавказский торгово-экономический техникум, 2020. – 51 с.

24. Алгоритмы. Основные алгоритмические конструкции [Текст]: сб. задач / сост. С.А. Рогозин. – Челябинск: Изд-во Челяб. гос. пед. ун-та, 2008. 42 с.

25. Глухова Л.А. Основы алгоритмизации и программирования : лаб. практикум для студ. спец. I-40 01 01 «Программное обеспечение информационных технологий» дневн. формы обуч. В 4 ч. Ч. 3 / Л.А. Глухова, Е П. Фадеева, Е.Е. Фадеева. – Минск : БГУИР, 2007. – 52 с.

26. Макконнелл Дж. Основы современных алгоритмов. 2-е дополненное издание. Москва: Техносфера, 2004. – 368с.

27. Орлов С. А. Теория и практика языков программирования: Учебник для вузов. Стандарт 3-го поколения. – СПб.: Питер. 2013. – 688 с.

28. Анисимов А.Е. Ведение в алгоритмизацию задач / А.Е. Анисимов. – Ижевск: Изд-во «Удмуртский университет», 2017. – 44 с.

29. Конова Е. А. Алгоритмы и программы. Язык С++ : учебное пособие для СПО / Е. А. Конова, Г. А. Поллак. – 3-е изд., стер. – Санкт-Петербург : Лань, 2022. – 384 с.

30. Паклина В.М., Паклина Е.М. Основы алгоритмизации и программирования. – Екатеринбург: ГОУ ВПО УГТУ−УПИ, 2009. – 91 с.

31. Ворожцов А.В., Винокуров Н.А. Лекции Алгоритмы: Построение, анализ и реализация на языке программирования Си. – Москва: МФТИ, 2007. – 452 с.

32. Мусатов Д.В. Сложность вычислений : конспект лекций. – Москва: МФТИ, 2017. – 224 с.

33. Седжвик Р. Фундаментальные алгоритмы на С++. Анализ / Структуры данных / Сортировка / Поиск : Пер. с англ. / Роберт Седжвик. – К.: Издательство «ДиаСофт», 2001. – 688 с.

34. Имран А. 40 алгоритмов, которые должен знать каждый программист на Python. – СПб.: Питер. 2023. – 368 с.

35. Hetland M. L. Python Algorithms: Mastering Basic Algorithms in the Python Language. – 2010. – 337 p.

36. Пинчук В.П. Анализ и построение алгоритмов. Краткий конспект лекций для спец. КСС. – Запорожье: ЗНТУ, 2012.

37. Филиповский В.М. Основы программирования и алгоритмизации. Ч. 2. Практикум по алгоритмизации: Учебное пособие. – СПб.: СПбПУ. 2022. – 71 с.

38. Кормен Т. Х. Алгоритмы: вводный курс. : Пер. с англ. – М. : ООО «И.Д. Вильямс», 2014. – 208 с.

39. Стивенс Р. Алгоритмы. Теория и практическое применение / Род Стивене. – Москва : Издательство «Э». 2016. – 544 с.